第2章 MySQL的调控按钮——启动选项和系统变量

2.1 启动选项和配置文件

大家应该都在手机中发现过一个“设置”功能，通过这个功能可以设置手机的来电铃声、音量大小、解锁密码等。假如没有这个设置功能，我们的生活将置于尴尬的境地。比如，在图书馆里无法把手机设置为静音，无法把流量开关关掉以节省流量，在别人得知解锁密码后无法更改密码。MySQL的服务器程序和客户端程序也有很多设置项，比如对于MySQL服务器程序，我们可以指定允许同时连入的客户端数量、客户端和服务器的通信方式、表的默认存储引擎、查询缓存的大小等信息。对于MySQL客户端程序，我们之前已经见识过了，可以指定需要连接的服务器程序所在主机的主机名或IP地址、用户名及密码等信息。

这些设置项一般都有各自的默认值，比如服务器允许同时连入的客户端的默认数量是151，表的默认存储引擎是InnoDB。我们可以在程序启动的时候修改这些默认值，对于这种在程序启动时指定的设置项也称之为启动选项（startup option），这些选项控制着程序启动后的行为。在MySQL安装目录的bin目录下的各种可执行文件，无论是服务器相关的程序（比如mysqld、mysqld_safe）还是客户端相关的程序（比如mysql、mysqladmin），在启动时基本都可以指定启动选项。这些启动选项可以在命令行中指定，也可以在配置文件中指定。

下面我们以mysqld为例，来详细唠叨一下指定启动选项的格式。

2.1.1 在命令行上使用选项

前文说过，服务器进程和客户端进程之间的通信有多种形式。如果我们想在启动服务器程序时就禁止各客户端使用TCP/IP网络进行通信，可以在启动服务器程序的命令行中添加skip-networking启动选项，就像下面这样：

```
mysqld --skip-networking
```

可以看到，在命令行中指定启动选项时需要在选项名前加上--前缀。另外，如果选项名是由多个单词构成的，它们之间可以由短划线-连接，也可以使用下划线_连接，也就是说skip-networking和skip_networking表示的含义是相同的。上面的写法与下面的写法是等价的：

```
mysqld --skip_networking
```

在按照上述命令启动服务器程序后，如果再使用mysql来启动客户端程序，把服务器主机名指定为127.0.0.1（IP地址的形式）的话会显示连接失败：

```
mysql -h127.0.0.1 -uroot -p
Enter password:

ERROR 2003 (HY000): Can't connect to MySQL server on '127.0.0.1' (61)
```

启动客户端程序时，在 -h 参数后边紧跟服务器的 IP 地址，这就意味着客户端要求和服务器之间通过 TCP/IP 网络进行通信。而此时连接失败的结果也就意味着我们在启动服务器时指定的启动选项 skip-networking 生效了。

再举一个例子，我们前边说过，如果在创建表的语句中没有显式指定表的存储引擎，那就会默认使用 InnoDB 作为表的存储引擎。如果我们想改变表的默认存储引擎，可以在黑框框中输入下面这样的启动服务器的命令：

```
mysqld --default-storage-engine=MyISAM
```

我们现在就已经把表的默认存储引擎改为 MyISAM 了。在客户端程序连接到服务器程序后试着创建一个表：

```
mysql> CREATE TABLE default_storage_engine_demo(
    ->     i INT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

这个表定义语句中并没有明确指定表的存储引擎，在表创建成功后再看一下这个表的结构：

```
mysql> SHOW CREATE TABLE default_storage_engine_demo\G
*************************** 1. row ***************************
       Table: default_storage_engine_demo
Create Table: CREATE TABLE 'default_storage_engine_demo' (
  'i' int(11) DEFAULT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8
1 row in set (0.01 sec)
```

可以看到该表的存储引擎已经是 MyISAM 了，这说明我们配置的启动选项 default-storage-engine 生效了。

总结一下，在启动服务器程序的命令行后边指定启动选项的通用格式就是这样的：

```
--启动选项1[=值1] --启动选项2[=值2] ... --启动选项n[=值n]
```

也就是说，我们可以将各个启动选项写到一行中，每一个启动选项名称前边添加 --，各个启动选项之间使用空白字符隔开。对于不需要值的启动选项，比如 skip-networking，它们就不需要指定对应的值。对于需要指定值的启动选项，比如 default-storage-engine，则在指定这个启动选项的时候需要显式指定它的值，比如 InnoDB、MyISAM 什么的。在命令行中指定有值的启动选项时需要注意，选项名、=、选项值之间不可以有空白字符，比如写成下面这样就是不正确的：

```
mysqld --default-storage-engine = MyISAM
```

每个 MySQL 程序都支持许多不同的选项。大多数程序提供了一个 --help 选项，可以用来查看该程序支持的全部启动选项以及它们的默认值。例如，使用 mysql --help 可以看到 mysql 程序支持的启动选项；使用 mysqld_safe --help 可以看到 mysqld_safe 程序支持的启动选项。不过查看 mysqld 支持的启动选项有些特别，需要使用 mysqld --verbose --help。

选项的长形式和短形式

我们前面提到的 skip-networking、default-storage-engine 这些启动选项都是长形式的选项（因为它们很长），设计 MySQL 的大叔为了方便我们使用，对于一些常用的选项提供了短形式。我们列举一些具有短形式的启动选项来瞅瞅（MySQL 支持的短形式选项太多了，篇幅所限，这里不全部列出），如表 2-1 所示。

表 2-1 选项的长形式、短形式及其含义

长形式	短形式	含义
--host	-h	主机名
--user	-u	用户名
--password	-p	密码
--port	-P	端口
--version	-V	版本信息

短形式的选项名只有一个字母，与使用长形式选项时需要在选项名前加两个短划线 -- 不同的是，使用短形式选项时在选项名前只加一个短划线 - 前缀。有一些短形式的选项之前已经接触过了，比如我们在启动服务器程序时通过添加短形式的选项 -P 来指定监听的端口号：

```
mysqld -P3307
```

使用短形式选项时，选项名和选项值之间可以没有间隙，也可以用空白字符隔开（-p 选项有些特殊，-p 和密码值之间不能有空白字符）。也就是说上面的命令和下面的是等价的：

```
mysqld -P 3307
```

另外，选项名是区分大小写的，比如 -p 和 -P 选项拥有完全不同的含义，大家需要注意。

2.1.2 配置文件中使用选项

在命令行中设置的启动选项只对当次启动生效，也就是说如果下一次重启程序的时候我们还想保留这些启动选项，还得重复把这些选项写到启动命令行中，这样真的神烦！于是设计 MySQL 的大叔提出了一个配置文件（也称为选项文件）的概念，我们把需要设置的启动选项都写在这个配置文件中，每次启动服务器时都从这个文件中加载相应的启动选项。由于这个配置文件可以长久地保存在计算机的硬盘中，所以我们只需配置一次，以后就不用显式地把启动选项都写在启动命令行中了。所以推荐使用配置文件的方式来设置启动选项。

1. 配置文件的路径

MySQL 程序在启动时会在多个路径下寻找配置文件，这些路径有的是固定的，有的可以在命令行中指定。根据操作系统的不同，寻找配置文件的路径也有所不同，我们分别看一下。

Windows 操作系统的配置文件

在 Windows 操作系统中，MySQL 会按照表 2-2 所示的路径依次寻找配置文件。

表 2-2 Windows 操作系统中配置文件的路径

路径名	备注
%WINDIR%\my.ini, %WINDIR%\my.cnf	
C:\my.ini, C:\my.cnf	
BASEDIR\my.ini, BASEDIR\my.cnf	
defaults-extra-file	命令行指定的额外配置文件路径
%APPDATA%\MySQL\.mylogin.cnf	登录路径选项（仅限客户端）

在阅读 Windows 操作系统下的这些配置文件路径时，需要注意下面这些事情。

- 在给定的前 3 个路径中，配置文件可以使用 .ini 的扩展名，也可以使用 .cnf 的扩展名。
- %WINDIR% 指的是你的机器上 Windows 目录的位置，通常是 C:\WINDOWS。如果不确定，可以使用 echo %WINDIR% 命令来查看。
- BASEDIR 指的是 MySQL 安装目录的路径，在我的 Windows 机器上，BASEDIR 的值是 C:\Program Files\MySQL\MySQL Server 5.7\。
- 第四个路径指的是在启动程序时可以通过指定 defaults-extra-file 启动选项的值来添加额外的配置文件路径。比如，我们在命令行中可以这么写：

```
mysqld --defaults-extra-file=C:\Users\xiaohaizi4919\my_extra_file.txt
```

  这样 MySQL 服务器在启动时就可以额外在 C:\Users\xiaohaizi4919\my_extra_file.txt 路径下查找配置文件。
- %APPDATA% 表示 Windows 应用程序数据目录的值，可以使用 echo %APPDATA% 命令查看。
- 表 2-2 中最后一个名为 .mylogin.cnf 的配置文件有点儿特殊，它不是一个纯文本文件（其他的配置文件都是纯文本文件），而是使用 mysql_config_editor 实用程序创建的加密文件。这个文件只能包含一些在启动客户端程序时用于连接服务器的选项，包括 host、user、password、port 和 socket，而且它只能被客户端程序所使用。

小贴士

mysql_config_editor 实用程序其实是 MySQL 安装目录的 bin 目录下的一个可执行文件，这个实用程序有专用的语法来生成或修改 .mylogin.cnf 文件中的内容。如何使用这个程序不是我们讨论的主题，大家可以到 MySQL 的官方文档中查看。

类 UNIX 操作系统中的配置文件

在类 UNIX 操作系统中，MySQL 会按照表 2-3 所示的路径来依次寻找配置文件。

表 2-3 类 UNIX 操作系统中配置文件的路径

路径名	备注
/etc/my.cnf	
/etc/mysql/my.cnf	
SYSCONFDIR/my.cnf	

续表

路径名	备注
$MYSQL_HOME/my.cnf	特定于服务器的选项（仅限服务器）
defaults-extra-file	命令行指定的额外配置文件路径
~/.my.cnf	特定于用户的选项
~/.mylogin.cnf	特定于用户的登录路径选项（仅限客户端）

同样，在阅读类 UNIX 操作系统下的这些配置文件路径时，需要注意下面这些事情。

- SYSCONFDIR 表示在使用 CMake 构建 MySQL 时使用 SYSCONFDIR 选项指定的目录。

小贴士

如果你不懂啥是 CMake，啥是编译，那就跳过吧，这对理解后续的文章没啥影响。

- MYSQL_HOME 是一个环境变量，该变量的值是我们自己设置的（想设置就设置，不想设置就不设置）。该变量的值代表一个路径，我们可以在该路径下创建一个 my.cnf 配置文件，这个配置文件中只能放置与启动服务器程序相关的选项（.mylogin.cnf 只能存放客户端相关的一些选项，除 .mylogin.cnf 以及 $MySQL_HOME/my.cnf 配置文件外，其余配置文件既可以存放服务器相关的选项，也可以存放客户端相关的选项）。

小贴士

如果使用 mysqld_safe 启动服务器程序，而且我们也没有主动设置这个 MySQL_HOME 环境变量的值，那么这个环境变量的值将自动被设置为 MySQL 的安装目录，也就是 MySQL 服务器将会在安装目录下查找名为 my.cnf 的配置文件。

- 表 2-3 中最后两个以 ~ 开头的路径是用户相关的。类 UNIX 系统中都有一个当前登录用户的概念，每个用户都可以有一个用户目录，~ 就代表这个用户目录。大家可以查看 HOME 环境变量的值来确定当前用户的用户目录。比如，我的 macOS 机器上的用户目录就是 /Users/xiaohaizi4919。之所以说表 2-3 中最后两个配置文件是用户相关的，是因为不同的类 UNIX 系统的用户都可以在自己的用户目录下创建 .my.cnf 或者 .mylogin.cnf。换句话说，不同登录用户使用的 .my.cnf 或者 .mylogin.cnf 配置文件是不同的。
- defaults-extra-file 的含义与 Windows 中的一样，不再赘述。
- .mylogin.cnf 的含义也同 Windows 中的一样。再次强调一遍，它不是纯文本文件，只能使用 mysql_config_editor 实用程序去创建或修改，用于存放客户端登录服务器时的相关选项。

总之，在我的计算机中，这几个路径中的任意一个都可以当作配置文件来使用。如果它们不存在，可以手动创建一个。比如，在 ~/.my.cnf 路径下手动创建一个配置文件。

另外，我们在唠叨如何启动 MySQL 服务器程序的时候说过，使用 mysqld_safe 程序启动服务器时，会调用 mysqld。对于传递给 mysqld_safe 的启动选项来说，如果 mysqld_safe 程序不处理，则会传递给 mysqld 程序处理。比如，skip-networking 选项是由 mysqld 处理的，mysqld_safe 并不处理，但是如果我们在命令行上执行下面的命令：

```
mysqld_safe --skip-networking
```

则在 mysqld_safe 调用 mysqld 时，会把它处理不了的这个 skip-networking 选项交给 mysqld 处理。

2. 配置文件的内容

与在命令行中指定启动选项不同的是，配置文件中的启动选项被划分为若干个组，每个组有一个组名，用中括号 [] 扩起来，像下面这样：

```
[server]
(具体的启动选项...)

[mysqld]
(具体的启动选项...)

[mysqld_safe]
(具体的启动选项...)

[client]
(具体的启动选项...)

[mysql]
(具体的启动选项...)

[mysqladmin]
(具体的启动选项...)
```

上面这个配置文件里就定义了许多个组，组名分别是 server、mysqld、mysqld_safe、client、mysql、mysqladmin。每个组下边可以定义若干个启动选项。我们以 [server] 组为例来看一下填写启动选项的形式（其他组中启动选项的形式是一样的）：

```
[server]
option1                 #这是option1，该选项不需要选项值
option2 = value2        #这是option2，该选项需要选项值
...
```

在配置文件中指定启动选项的语法类似于命令行语法，但是在配置文件中只能使用长形式的选项，而且在配置文件中指定的启动选项不允许加 -- 前缀，并且每行只指定一个选项，等号 = 周围可以有空白字符（在命令行中，选项名、=、选项值之间不允许有空白字符）。另外，在配置文件中，我们可以使用 # 来添加注释，从 # 出现直到行尾的内容都属于注释内容，MySQL 程序会忽略这些注释内容。

为了让大家更容易对比在命令行和配置文件中指定启动选项的区别，我们再把在命令行中指定 option1 和 option2 两个选项的格式写一遍看看：

```
--option1 --option2=value2
```

在配置文件中，不同的选项组是给不同的程序使用的。如果选项组名称与程序名称相同，则组中的选项将专门应用于该程序。例如，[mysqld] 和 [mysql] 组分别应用于 mysqld 服务器程序和 mysql 客户端程序。不过有两个选项组比较特别：

- [server] 组下面的启动选项将作用于所有的服务器程序；
- [client] 组下面的启动选项将作用于所有的客户端程序。

需要注意的一点是，mysqld_safe 和 mysql.server 这两个程序在启动时都会读取 [mysqld] 选项组中的内容。为了直观感受一下，我们挑一些程序来看看它们能读取的选项组都有哪些（见表 2-4）。

表 2-4 程序的对应类别和能读取的组

程序名	类别	能读取的组
mysqld	启动服务器	[mysqld]、[server]
mysqld_safe	启动服务器	[mysqld]、[server]、[mysqld_safe]
mysql.server	启动服务器	[mysqld]、[server]、[mysql.server]
mysql	启动客户端	[mysql]、[client]
mysqladmin	启动客户端	[mysqladmin]、[client]
mysqldump	启动客户端	[mysqldump]、[client]

现在以 macOS 操作系统为例，在 /etc/mysql/my.cnf 配置文件中添加一些内容（如果大家使用的是 Windows 系统，请自行参考前文提到的配置文件路径）：

```
[server]
skip-networking
default-storage-engine=MyISAM
```

然后直接用 mysqld 启动服务器程序：

```
mysqld
```

虽然在命令行中没有添加启动选项，但是在程序启动时，会默认地到我们上面提到的配置文件路径下查找配置文件，其中就包括 /etc/mysql/my.cnf。又由于 mysqld 命令可以读取 [server] 选项组的内容，所以 skip-networking 和 default-storage-engine=MyISAM 这两个选项是生效的。大家可以把这些启动选项放在 [client] 组中，然后再试试用 mysqld 启动服务器程序，看看里面的启动选项是否生效（剧透一下，不生效）。

3. 特定 MySQL 版本的专用选项组

我们可以在选项组的名称后加上特定的 MySQL 版本号。比如对于 [mysqld] 选项组来说，我们可以定义一个 [mysqld-5.7] 的选项组。它的含义和 [mysqld] 一样，只不过只有版本号为 5.7 的 mysqld 程序才能使用这个选项组中的选项。

4. 配置文件的优先级

我们前面唠叨过，MySQL 将在某些固定的路径下搜索配置文件。我们也可以通过在命令行中指定 defaults-extra-file 启动选项来指定额外的配置文件路径。MySQL 将按照表 2-2 或表 2-3 中给定的顺序（具体取决于所用的操作系统）依次读取各个配置文件。如果该文件不存在，则

忽略。值得注意的是，如果我们在多个配置文件中设置了相同的启动选项，则以最后一个配置文件中的为准。比如 /etc/my.cnf 文件的内容是这样的：

```
[server]
default-storage-engine=InnoDB
```

而 ~/.my.cnf 文件中的内容是这样的：

```
[server]
default-storage-engine=MyISAM
```

又因为 ~/.my.cnf 比 /etc/my.cnf 顺序靠后，因此，若两个配置文件中出现相同的启动选项，将以 ~/.my.cnf 中的为准。所以，在 MySQL 服务器程序启动之后，default-storage-engine 的值就是 MyISAM。

5. 同一个配置文件中多个组的优先级

我们说同一个程序可以访问配置文件中的多个组，比如mysqld可以访问 [mysqld]、[server] 组。如果在同一个配置文件中（比如 ~/.my.cnf），在 [mysqld]、[server] 组里出现了同样的启动选项，比如下面这样：

```
[server]
default-storage-engine=InnoDB

[mysqld]
default-storage-engine=MyISAM
```

那么，将以最后一个出现的组中的启动选项为准。比如，在上面的例子中，default-storage-engine 既出现在 [server] 组也出现在 [mysqld] 组，由于 [mysqld] 组在 [server] 组后边，所以将以 [mysqld] 组中的配置项为准。

6. defaults-file 的使用

如果我们不想让 MySQL 到默认的路径下搜索配置文件，则可以在命令行指定 defaults-file 选项，比如下面这样（以类 UNIX 系统为例）：

```
mysqld --defaults-file=/tmp/myconfig.txt
```

这样一来，在程序启动时将只在 /tmp/myconfig.txt 路径下搜索配置文件。如果文件不存在或无法访问，则会发生错误。

小贴士

注意 defaults-extra-file 和 defaults-file 的区别，使用 defaults-extra-file 可以指定额外的配置文件路径（也就是说那些固定的配置文件路径也会被搜索）。

2.1.3 在命令行和配置文件中启动选项的区别

在命令行中指定的绝大部分启动选项都可以放到配置文件中，但是有一些选项是专门为命令行设计的，比如 defaults-extra-file、defaults-file 这样的选项本身就是为了指定配置文件路径

的，如果再放在配置文件中使用就没啥意义了。剩下的一些只能用到命令行中而不能用到配置文件中的启动选项就不一一列举了，等到用的时候再提（本书中用不到，有兴趣的读者请移步到官方文档）。

另外有一点需要特别注意：如果同一个启动选项既出现在命令行中，又出现在配置文件中，那么以命令行中的启动选项为准！比如我们在配置文件中写了：

```
[server]
default-storage-engine=InnoDB
```

而我们的启动命令是：

```
mysqld --default-storage-engine=MyISAM
```

那么，最后 default-storage-engine 的值就是 MyISAM！

2.2 系统变量

2.2.1 系统变量简介

MySQL 服务器程序在运行过程中会用到许多影响程序行为的变量，它们被称为系统变量。比如，允许同时连入的客户端数量用系统变量 max_connections 表示；表的默认存储引擎用系统变量 default_storage_engine 表示；查询缓存的大小用系统变量 query_cache_size 表示。MySQL 服务器程序的系统变量有好几百个，这里不再一一列举。每个系统变量都有一个默认值，我们可以使用命令行或者配置文件中的选项在启动服务器时改变一些系统变量的值。大多数系统变量的值也可以在程序运行过程中修改，而无须停止并重新启动服务器。

2.2.2 查看系统变量

我们可以使用下列命令查看 MySQL 服务器程序支持的系统变量以及它们的当前值：

```
SHOW VARIABLES [LIKE 匹配的模式];
```

由于系统变量实在太多了，如果我们直接使用 SHOW VARIABLES 查看的话，就直接在屏幕上刷屏了，所以通常都会使用一个 LIKE 表达式来指定过滤条件，比如这么写：

```
mysql> SHOW VARIABLES LIKE 'default_storage_engine';
+-------------------------+---------+
| Variable_name           | Value   |
+-------------------------+---------+
| default_storage_engine | InnoDB |
+-------------------------+---------+
1 row in set (0.01 sec)

mysql> SHOW VARIABLES like 'max_connections';
+-----------------+-------+
| Variable_name   | Value |
```

```
+-----------------+-------+
| max_connections | 151   |
+-----------------+-------+
1 row in set (0.00 sec)
```

可以看到，现在服务器程序使用的默认存储引擎就是InnoDB，允许同时连接的客户端数量最多为151。

小贴士

更严谨地说，MySQL服务器实际上允许max_connections + 1个客户端连接，额外的1个是给超级用户准备的（很显然这是超级用户的一个特权）。

别忘了LIKE表达式中可以使用通配符来进行模糊查询，也就是说我们可以这么写：

```
mysql> SHOW VARIABLES LIKE 'default%';
+-------------------------------+-----------------------+
| Variable_name                 | Value                 |
+-------------------------------+-----------------------+
| default_authentication_plugin | mysql_native_password |
| default_password_lifetime     | 0                     |
| default_storage_engine        | InnoDB                |
| default_tmp_storage_engine    | InnoDB                |
| default_week_format           | 0                     |
+-------------------------------+-----------------------+
5 rows in set (0.01 sec)
```

这样就查出了所有以default开头的系统变量的值。

2.2.3 设置系统变量

1. 通过启动选项设置

大部分系统变量都可以通过在启动服务器时传送启动选项的方式来设置。如何填写启动选项我们在前面已经花了大量篇幅来唠叨了，其实就是下面这两种方式：

- 通过命令行添加启动选项。

  比方说在启动服务器程序时用这个命令：

  ```
  mysqld --default-storage-engine=MyISAM --max-connections=10
  ```

- 通过配置文件添加启动选项。

  可以这样填写配置文件：

  ```
  [server]
  default-storage-engine=MyISAM
  max-connections=10
  ```

  当使用上面的任何一种方式启动服务器程序后，再来看一下系统变量的值：

```
mysql> SHOW VARIABLES LIKE 'default_storage_engine';
+-------------------------+--------+
| Variable_name           | Value  |
```

```
+-------------------------+--------+
| default_storage_engine | MyISAM |
+-------------------------+--------+
1 row in set (0.00 sec)

mysql> SHOW VARIABLES LIKE 'max_connections';
+-----------------+-------+
| Variable_name   | Value |
+-----------------+-------+
| max_connections | 10    |
+-----------------+-------+
1 row in set (0.00 sec)
```

可以看到 default_storage_engine 和 max_connections 这两个系统变量的值已经被修改了。需要注意的一点是，对于启动选项来说，如果启动选项名由多个单词组成，各个单词之间用短划线（-）或者下划线（_）连接起来都可以；但是对于对应的系统变量来说，各个单词之间必须使用下划线（_）连接起来。

2. 服务器程序运行过程中设置

对于大部分系统变量来说，它们的值可以在服务器程序运行过程中进行动态修改而无须停止并重启服务器。不过系统变量有作用范围之分，下面详细唠叨一下。

（1）设置不同作用范围的系统变量

我们前面说过，多个客户端程序可以同时连接到一个服务器程序。对于同一个系统变量，我们有时想让不同的客户端有不同的值。比方说狗哥使用客户端 A，他想让当前客户端对应的默认存储引擎为 InnoDB，所以他可以把系统变量 default_storage_engine 的值设置为 InnoDB；猫爷使用客户端 B，他想让当前客户端对应的默认存储引擎为 MyISAM，所以他可以把系统变量 default_storage_engine 的值设置为 MyISAM。这样可以使狗哥和猫爷的的客户端拥有不同的默认存储引擎，且在使用时互不影响，十分方便。但是，这样一来各个客户端都私有一份系统变量，这会产生两个问题。

- 有一些系统变量并不是针对单个客户端的，比如允许同时连接到服务器的客户端数量 max_connections、查询缓存的大小 query_cache_size，这些公有的系统变量让某个客户端私有显然不合适。
- 一个新客户端连接到服务器时，与它对应的系统变量的值该怎么设置。

为了解决这两个问题，设计 MySQL 的大叔提出了系统变量的作用范围的概念。具体来说，作用范围分为下面两种。

- GLOBAL（全局范围）：影响服务器的整体操作。具有 GLOBAL 作用范围的系统变量可以称为全面变量。
- SESSION（会话范围）：影响某个客户端连接的操作。具有 SESSION 作用范围的系统变量可以称为会话变量。

服务器在启动时，会将每个全局变量初始化为其默认值（可以通过命令行或配置文件中指定的选项更改这些默认值）。服务器还为每个连接的客户端维护一组会话变量，客户端的会话变量在连接时使用相应全局变量的当前值进行初始化（也有一些会话变量不依据相应的全局变量值进行初始化，不过这里不展开唠叨了）。

这话有点儿绕，还是以 default_storage_engine 为例来解释。在服务器启动时会初始化一

个名为default_storage_engine、作用范围为GLOBAL的系统变量。之后每当有一个客户端连接到该服务器时，服务器都会单独为该客户端分配一个名为default_storage_engine、作用范围为SESSION的系统变量，这个作用范围为SESSION的系统变量值按照当前作用范围为GLOBAL的同名系统变量值进行初始化。

很显然，通过启动选项设置的系统变量的作用范围都是GLOBAL的，因为在服务器启动的时候还没有客户端程序连接进来呢。了解了系统变量的GLOBAL和SESSION作用范围之后，我们再看一下在服务器程序运行期间通过客户端程序设置系统变量的语法：

```
SET [GLOBAL|SESSION] 系统变量名 = 值;
```

或者写成这样也行：

```
SET [@@(GLOBAL|SESSION).]系统变量名 = 值;
```

比如，我们想在服务器的运行过程中把作用范围为GLOBAL的系统变量default_storage_engine的值修改为MyISAM，也就是想让之后新连接到服务器的客户端都用MyISAM作为默认的存储引擎，则可以选择下面两条语句中的任意一条来设置。

- 语句1：SET GLOBAL default_storage_engine = MyISAM;
- 语句2：SET @@GLOBAL.default_storage_engine = MyISAM;

如果只想对本客户端生效，也可以选择下面3条语句中的任意一条来设置。

- 语句1：SET SESSION default_storage_engine = MyISAM;
- 语句2：SET @@SESSION.default_storage_engine = MyISAM;
- 语句3：SET default_storage_engine = MyISAM;

从上面的语句3也可以看出，如果在设置系统变量的语句中省略了作用范围，默认的作用范围就是SESSION。也就是说“SET系统变量名=值”和“SET SESSION系统变量名=值”是等价的。

（2）查看不同作用范围的系统变量

我们可以在查看系统变量的语句中加上要查看哪个作用范围的系统变量的修饰符，就像下面这样：

```
SHOW [GLOBAL|SESSION] VARIABLES [LIKE 匹配的模式];
```

- 如果使用GLOBAL修饰符，则显示全局系统变量的值。如果某个系统变量没有GLOBAL作用范围，则不显示它。
- 如果使用SESSION修饰符，则显示针对当前连接有效的系统变量值。如果某个系统变量没有SESSION作用范围，则显示GLOBAL作用范围的值。
- 如果没写修饰符，则与使用SESSION修饰符效果一样。

下面演示一下完整地设置并查看系统变量的过程：

```
mysql> SHOW SESSION VARIABLES LIKE 'default_storage_engine';
+-------------------------+--------+
| Variable_name           | Value  |
+-------------------------+--------+
| default_storage_engine | InnoDB |
+-------------------------+--------+
```

```
1 row in set (0.00 sec)

mysql> SHOW GLOBAL VARIABLES LIKE 'default_storage_engine';
+-------------------------+--------+
| Variable_name           | Value  |
+-------------------------+--------+
| default_storage_engine | InnoDB |
+-------------------------+--------+
1 row in set (0.00 sec)

mysql> SET SESSION default_storage_engine = MyISAM;
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW SESSION VARIABLES LIKE 'default_storage_engine';
+-------------------------+--------+
| Variable_name           | Value  |
+-------------------------+--------+
| default_storage_engine | MyISAM |
+-------------------------+--------+
1 row in set (0.00 sec)

mysql> SHOW GLOBAL VARIABLES LIKE 'default_storage_engine';
+-------------------------+--------+
| Variable_name           | Value  |
+-------------------------+--------+
| default_storage_engine | InnoDB |
+-------------------------+--------+
1 row in set (0.00 sec)
```

可以看到，最初 default_storage_engine 的系统变量无论是在 GLOBAL 作用范围还是在 SESSION 作用范围，值都是 InnoDB。我们把 SESSION 作用范围的系统变量值设置为 MyISAM 之后，可以看到 GLOBAL 作用范围的值并没有改变。

小贴士

如果某个客户端改变了某个系统变量在 GLOBAL 作用范围的值，并不会影响该系统变量在当前已经连接的客户端作用范围为 SESSION 的值，只会影响后续连入的客户端作用范围为 SESSION 的值。

（3）注意事项

- 并不是所有的系统变量都具有 GLOBAL 和 SESSION 的作用范围。
  - 有一些系统变量只具有 GLOBAL 作用范围，比如 max_connections，它表示服务器程序支持同时最多有多少个客户端程序进行连接。
  - 有一些系统变量只具有 SESSION 作用范围，比如 insert_id，它表示在对某个包含 AUTO_INCREMENT 列的表进行插入时，该列初始的值。
  - 有一些系统变量的值既具有 GLOBAL 作用范围，也具有 SESSION 作用范围，比如前面用到的 default_storage_engine，而且其实大部分的系统变量都是这样的。
- 有些系统变量是只读的，并不能设置值。

  比如 version，它表示当前 MySQL 的版本。客户端不能设置它的值，只能在 SHOW VARIABLES 语句中查看。

3. 启动选项和系统变量的区别

启动选项是在程序启动时由用户传递的一些参数，而系统变量是影响服务器程序运行行为的变量。它们之间的关系如下。

- 大部分的系统变量都可以当作启动选项传入。
- 有些系统变量是在程序运行过程中自动生成的，不可以当作启动选项来设置，比如 character_set_client。
- 有些启动选项也不是系统变量，比如 defaults-file。

2.3 状态变量

为了让我们更好地了解服务器程序的运行情况，MySQL 服务器程序中维护了好多关于程序运行状态的变量，它们被称为状态变量。比如，Threads_connected 表示当前有多少客户端与服务器建立了连接；Innodb_rows_updated 表示更新了多少条以 InnoDB 为存储引擎的表中的记录。像这样显示服务器程序状态信息的状态变量还有好几百个，我们就不一一唠叨了，等遇到时再详细说明它们的作用。

由于状态变量是用来显示服务器程序运行状态的，所以它们的值只能由服务器程序自己来设置，不能人为设置。与系统变量类似，状态变量也有 GLOBAL 和 SESSION 两个作用范围，查看状态变量的语句可以这么写：

```
SHOW [GLOBAL|SESSION] STATUS [LIKE 匹配的模式];
```

类似地，如果不写修饰符，则与使用 SESSION 修饰符效果一样。

我们看一下所有以 Thread 开头的状态变量的值都是什么：

```
mysql> SHOW STATUS LIKE 'thread%';
+-------------------+-------+
| Variable_name     | Value |
+-------------------+-------+
| Threads_cached    | 0     |
| Threads_connected | 1     |
| Threads_created   | 1     |
| Threads_running   | 1     |
+-------------------+-------+
4 rows in set (0.00 sec)
```

2.4 总结

启动选项可以调整服务器启动后的一些行为。它们可以在命令行中指定，也可以将它们写入配置文件中。

在命令行中指定启动选项时，可以将各个启动选项写到一行中，每一个启动选项名称前面添加 --，而且各个启动选项之间使用空白字符隔开。有一些启动选项不需要指定选项值，有一些选项需要指定选项值。在命令行中指定有值的启动选项时需要注意，选项名、=、选项值之

间不可以有空白字符。一些常用的启动选项具有短形式的选项名，使用短形式选项时在选项名前只加一个短划线 - 前缀。

服务器程序在启动时将会在一些给定的路径下搜索配置文件，不同操作系统的搜索路径是不同的。

配置文件中的启动选项被划分为若干个组，每个组有一个组名，用中括号 [] 扩起来。在配置文件中指定的启动选项不允许添加 -- 前缀，并且每行只指定一个选项，而且等号 = 周围可以有空白字符。我们可以使用 # 来添加注释。

系统变量是服务器程序中维护的一些变量，这些变量影响着服务器的行为。修改系统变量的方式如下。

- 在服务器启动时通过添加相应的启动选项进行修改。
- 在运行时使用 SET 语句修改，下面两种方式都可以：
  - SET [GLOBAL|SESSION] 系统变量名 = 值 ;
  - SET [@@(GLOBAL|SESSION).] 系统变量名 = 值 ;

查看系统变量的方式如下所示：

```
SHOW [GLOBAL|SESSION] VARIABLES [LIKE 匹配的模式];
```

状态变量是用来显示服务器程序运行状态的，我们可以使用下面的命令来查看，而且只能查看：

```
SHOW [GLOBAL|SESSION] STATUS [LIKE 匹配的模式];
```